

Air University



OOP (Lab)

Year 2025

Mr. Jahangir Anwar

Academic Year: 2024 – 2028
Department of Cyber Security

ZAEEM FURQAN | 247169
USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: June 10, 2025

Event Management System

Code

- Includes essential C++ standard libraries for I/O, memory, containers, and utilities.
- Uses using namespace std to avoid prefixing standard library names.
- Forward declares the Event class for use before its full definition.
- Declares global events vector to store dynamically created Event objects via *unique_ptr*.
- Declares global *selectedVenues* vector to store dynamically created Venue objects via *unique_ptr*.
- Defines a constant list *fixedVenues* containing venue names with their seating capacities.
- The list includes a variety of predefined venues like "Auditorium", "Ballroom", and "Computer Lab".

```
1  #include <iostream>
2  #include <string>
3  #include <cstring>
4  #include <vector>
5  #include <memory>
6  #include <regex>
7  #include <ctime>
8
9  using namespace std;
10
11 // Forward declaration
12 class Event;
13
14 // Global container for events
15 vector<unique_ptr<Event>> events;
16 // Global container for user-selected venues
17 vector<unique_ptr<class Venue>> selectedVenues;
18
19 // Fixed venues list
20 const vector<pair<string, int>> fixedVenues = {
21     {"Auditorium", 200},
22     {"Sports Complex", 1000},
23     {"Grand Hall", 800},
24     {"Ballroom", 500},
25     {"Hotel Banquet Hall", 1000},
26     {"Grand Dining Hall", 400},
27     {"Lecture Theatre", 300},
28     {"Conference Room", 500},
29     {"Seminar Room", 100},
30     {"Departmental Auditorium", 150},
31     {"Classroom", 35},
32     {"Computer Lab", 40},
33     {"Exhibition Space", 300},
34     {"Marquee/Outdoor Event Space", 500}};
35
```

- Defines the **Venue** class to represent event venues.
- Private members include a **unique id**, **static total_venues counter**, **name**, and **capacity**.
- Constructor initializes **id** using **total_venues++** and sets **name** and **capacity**.
- **getName()** and **getCapacity()** are accessor methods returning the venue's name and capacity.
- **displaydetails()** prints the venue's name and capacity to the console.
- **total_venues** is initialized to 0 outside the class definition.
- Ensures each Venue instance has a unique ID and consistent data display.

```

36 // ----- Venue -----
37 class Venue
38 {
39 private:
40     const int id;
41     static int total_venues;
42     string name;
43     int capacity;
44
45 public:
46     Venue(string n, int c) : id(total_venues++), name(n), capacity(c) {}
47
48     string getName() const { return name; }
49     int getCapacity() const { return capacity; }
50
51     void displaydetails() const
52     {
53         cout << "Venue Name: " << name << endl;
54         cout << "Capacity: " << capacity << endl;
55     }
56 };
57 int Venue::total_venues = 0;
58

```

- The **Organizer** class has a private string member **name** to store the organizer's name.
- It provides a constructor **Organizer(string n)** that initializes the name member using an initializer list.
- A **getName()** method is available to retrieve the organizer's name.
- The **create_organizer()** is a static method that prompts the user for an organizer's name and returns a **unique_ptr<Organizer>** to a newly created Organizer object, ensuring proper memory management.
- The **displaydetails()** method is virtual and prints the organizer's name to the console.
- A virtual destructor **~Organizer() = default** is declared, indicating that Organizer is intended to be a base class in a potential inheritance hierarchy.

```

59 // ----- Organizer -----
60 class Organizer
61 {
62 private:
63     string name;
64
65 public:
66     Organizer(string n) : name(n) {}
67
68     string getName() const { return name; }
69
70     static unique_ptr<Organizer> create_organizer()
71     {
72         string n;
73         cout << "Enter Organizer Name: ";
74         getline(cin, n);
75         return make_unique<Organizer>(n);
76     }
77
78     virtual void displaydetails() const
79     {
80         cout << "Organizer Name: " << name << endl;
81     }
82
83     virtual ~Organizer() = default;
84 };
85

```

- The *Attendee* class has a private **static int total_attendees** member, which keeps a count of all Attendee objects created.
- The default constructor **Attendee()** increments **total_attendees** each time a new *Attendee* object is created.
- The **create_attendee()** is a static method that returns a **unique_ptr<Attendee>** to a newly created Attendee object, ensuring proper memory management.
- The **get_total_attendees()** is a static method that returns the current count of **total_attendees**.
- Outside the class definition, **int Attendee::total_attendees = 0;** initializes the static member **total_attendees** to zero.

```

86 // ----- Attendee -----
87 class Attendee
88 {
89 private:
90     static int total_attendees;
91
92 public:
93     Attendee()
94     {
95         total_attendees++;
96     }
97
98     static unique_ptr<Attendee> create_attendee()
99     {
100         return make_unique<Attendee>();
101     }
102
103     static int get_total_attendees()
104     {
105         return total_attendees;
106     }
107 };
108 int Attendee::total_attendees = 0;
109

```

- The *Event* class has protected members: a **static int total_events** to count all events, a string date, and a **unique_ptr<Organizer>** organizer to manage an associated Organizer object.
- It defines a nested **struct VenueAttendees** to associate a **unique_ptr<Venue>** (presumably a Venue object) with an **int attendeeCount**.
- The Event class also has a **vector<VenueAttendees> venuesWithAttendees** to store multiple venue-attendee associations.
- It declares pure virtual functions **inputdetails()** and **displaydetails() const**, making Event an abstract base class, meaning it cannot be instantiated directly.
- A static method **create_event(int event_number)** is declared, suggesting a factory-like pattern for creating different types of events.

```

110 // ----- Event Base -----
111 class Event
112 {
113 protected:
114     static int total_events;
115     string date;
116     unique_ptr<Organizer> organizer;
117
118     // Store venues and corresponding attendees counts
119     struct VenueAttendees
120     {
121         unique_ptr<Venue> venue;
122         int attendeeCount;
123     };
124     vector<VenueAttendees> venuesWithAttendees;
125
126 public:
127     Event() {}
128
129     virtual void inputdetails() = 0;
130     virtual void displaydetails() const = 0;
131
132     static void create_event(int event_number); // defined later
133
134     virtual ~Event() = default;
135
136     // Utility functions for validation
137     static bool isValidName(const string &name)
138     {
139         return !name.empty();
140     }
141

```

- A virtual **~Event() = default** destructor is provided, confirming its role as a base class.

- A static utility function **isValidName(const string &name)** is included, which checks if a given string name is not empty.

```
142 static bool isValidDate(const string &date)
143 {
144     regex datePattern(R"(\d{2}-\d{2}-\d{4})");
145     if (!regex_match(date, datePattern))
146         return false;
147
148     int day = stoi(date.substr(0, 2));
149     int month = stoi(date.substr(3, 2));
150     int year = stoi(date.substr(6, 4));
151
152     // Check month range
153     if (month < 1 || month > 12)
154         return false;
155
156     // Check day range based on month
157     if (day < 1 || day > 31)
158         return false;
159
160     // Days in each month
161     const int daysInMonth[] = {0, 31, 28 + (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0)),
162                                31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
163     if (day > daysInMonth[month])
164         return false;
165
166     return true;
167 }
168
```

- The **isValidDate** method takes a **const string &date** as input and uses regular expressions to validate its format as "DD-MM-YYYY". If the format doesn't match, it returns false.
- It then parses the day, month, and year from the input date string using **stoi** and **substr**.
- It checks if the month is within a valid range (**1 to 12**); if not, it returns **false**.
- It then checks if the day is within a general valid range (**1 to 31**); if not, it returns **false**.
- An array **daysInMonth** is used to store the maximum number of days for each month, with a specific calculation for February to account for leap years (considering years divisible by **4**, but not by **100** unless also by **400**).
- Finally, it checks if the parsed day is greater than the valid number of days for its specific month (**using the daysInMonth array**); if so, it returns **false**. Otherwise, the date is considered valid, and the method returns **true**.

```
169 static bool isAtLeastOneWeekAhead(const string &date)
170 {
171     // Get current date
172     time_t t = time(0);
173     tm *now = localtime(&t);
174     now->tm_mday += 7; // Add 7 days
175     mktime(now);      // Normalize the date
176
177     // Convert to YYYY-MM-DD format for comparison
178     char currentDate[11];
179     strftime(currentDate, sizeof(currentDate), "%Y-%m-%d", now);
180
181     // Convert input date to YYYY-MM-DD format
182     int day = stoi(date.substr(0, 2));
183     int month = stoi(date.substr(3, 2));
184     int year = stoi(date.substr(6, 4));
185     char inputDate[11];
186     snprintf(inputDate, sizeof(inputDate), "%04d-%02d-%02d", year, month, day);
187
188     return strcmp(inputDate, currentDate) > 0; // Compare dates
189 }
190 };
191 int Event::total_events = 0;
192
```

- The *isAtLeastOneWeekAhead* method takes a **const string &date** as input and determines if it is at least one week ahead of the current date.
- It first obtains the current time using **time(0)** and converts it to a local time structure **tm** using **localtime()**.
- It then adds 7 days to the current day (**now->tm_mday += 7**) and normalizes the date using **mktime(now)** to handle potential month/year rollovers.
- The current date (plus 7 days) is then formatted into a "YYYY-MM-DD" string using **strftime** for consistent comparison.
- The input date string (assumed to be "DD-MM-YYYY") is parsed into day, month, and year integers.
- The parsed input date is then formatted into a "YYYY-MM-DD" string using **snprintf**.
- Finally, it compares the formatted input date string with the formatted current date string (plus 7 days) using **strcmp**. A return value greater than 0 indicates that the input date is later than the current date plus one week, thus returning true.
- Outside the class definition, **int Event::total_events = 0;** initializes the static member **total_events** for the Event class to zero.

```
193 // ----- Utility function to select venues -----
194 void select_venues()
195 {
196     cout << "\n===== Venue Selection =====\n";
197     cout << "Available Venues:\n";
198     for (size_t i = 0; i < fixedVenues.size(); ++i)
199     {
200         cout << i + 1 << ". " << fixedVenues[i].first << " (Capacity: " << fixedVenues[i].second << ")\n";
201     }
202
203     int n;
204     cout << "How many venues do you want to select (1-" << fixedVenues.size() << ")? ";
205     cin >> n;
206     while (n < 1 || n > (int)fixedVenues.size())
207     {
208         cout << "Invalid number. Please enter between 1 and " << fixedVenues.size() << ": ";
209         cin >> n;
210     }
211
212     vector<bool> chosen(fixedVenues.size(), false);
213     selectedVenues.clear();
214
215     for (int i = 0; i < n; ++i)
216     {
217         int choice;
218         cout << "Select Venue #" << (i + 1) << ": ";
219         cin >> choice;
220         while (choice < 1 || choice > (int)fixedVenues.size() || chosen[choice - 1])
221         {
222             if (choice >= 1 && choice <= (int)fixedVenues.size() && chosen[choice - 1])
223                 cout << "Venue already selected. Choose a different one: ";
224             else
225                 cout << "Invalid choice. Enter a number between 1 and " << fixedVenues.size() << ": ";
226             cin >> choice;
227         }
228         chosen[choice - 1] = true;
229         selectedVenues.push_back(make_unique<Venue>(fixedVenues[choice - 1].first, fixedVenues[choice - 1].second));
230     }
231     cin.ignore(); // clear input buffer
232 }
233
```

- The ***select_venues()*** function is designed to allow a user to select multiple venues from a predefined list.
- It first displays a list of "Available Venues", iterating through a **fixedVenues** container (presumably a vector of **pair<string, int>**, where the string is the venue name and the int is its capacity), and presents them with an index and their capacity.
- It then prompts the user to enter the number of venues they wish to select, validating the input to ensure it's within the valid range (1 to the total number of fixed venues).
- A **vector<bool>** **chosen** is initialized to keep track of already selected venues to prevent duplicates. The **selectedVenues** (presumably a member of the class containing this function) is cleared before new selections.
- The function then enters a loop for **n** iterations (where **n** is the number of venues the user wants to select). In each iteration, it prompts the user to select a venue by its number.
- Input validation for venue selection ensures the choice is within the valid range and that the venue has not been chosen already. If an invalid or already chosen venue is selected, an **error message** is displayed, and the user is prompted again.
- Once a valid and unchosen venue is selected, its corresponding entry in the **chosen** vector is marked as **true**, and a **unique_ptr** to a new **Venue** object (created from the selected venue's name and capacity) is added to the **selectedVenues** container.
- Finally, **cin.ignore()** is called, likely to clear the input buffer after numeric input, preventing issues with subsequent **getline()** calls.


```

234 // Drive class Convocation Ceremony
235 class ConvocationCeremony : public Event
236 {
237 public:
238     void inputdetails() override
239     {
240         cout << "\n----- Convocation Ceremony Details ----- \n";
241
242         while (true)
243         {
244             cout << "Enter Event Date (DD-MM-YYYY): ";
245             getline(cin, date);
246             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
247                 break;
248             cout << "Invalid format or date must be at least one week ahead.\n";
249         }
250
251         organizer = Organizer::create_organizer();
252         venuesWithAttendees.clear();
253         for (const auto &venue : selectedVenues)
254         {
255             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
256             int attendees;
257             while (true)
258             {
259                 cout << "Attendees at " << venueCopy->getName() << " (max " << venueCopy->getCapacity() << "): ";
260                 cin >> attendees;
261                 cin.ignore();
262                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
263                     break;
264                 cout << "Invalid count.\n";
265             }
266             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
267         }
268     }
269
270     void displaydetails() const override
271     {
272         cout << "\n----- Convocation Ceremony Summary ----- \n";
273         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
274         for (const auto &va : venuesWithAttendees)
275             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
276     }
277 };

```

- The **ConvocationCeremony** class inherits publicly from Event, indicating it's a specific type of event.
- It overrides the **inputdetails()** method to gather specific information for a convocation ceremony.
- Inside **inputdetails()**, it prompts the user to enter the event date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the Event class. It repeatedly prompts until a valid date is entered.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the organizer member inherited from Event.
- The **venuesWithAttendees** vector (also inherited from Event) is cleared.
- It iterates through **selectedVenues** (presumably a member containing pre-selected venues, possibly from a **select_venues()** call) to get attendee counts for each. For each venue, it prompts the user to enter the number of attendees, ensuring the count is non-negative and does not exceed the venue's capacity.
- After obtaining the attendee count for a venue, it creates a **VenueAttendees** object using **std::move** to transfer ownership of the **venueCopy** (a **unique_ptr** to a Venue object) and the **attendees** count, then pushes it into the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to show a summary of the convocation ceremony.
- Inside **displaydetails()**, it prints a "Convocation Ceremony Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name and its corresponding attendee count.


```

279 // Drive class Gala Dinner
280 class GalaDinner : public Event
281 {
282 public:
283     void inputdetails() override
284     {
285         cout << "\n----- Gala Dinner Details ----- \n";
286         while (true)
287         {
288             cout << "Enter Event Date (DD-MM-YYYY): ";
289             getline(cin, date);
290             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
291                 break;
292             cout << "Invalid format or date must be at least one week ahead.\n";
293         }
294         organizer = Organizer::create_organizer();
295         venuesWithAttendees.clear();
296         for (const auto &venue : selectedVenues)
297         {
298             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
299             int attendees;
300             while (true)
301             {
302                 cout << "Attendees at " << venueCopy->getName() << ": ";
303                 cin >> attendees;
304                 cin.ignore();
305                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
306                     break;
307                 cout << "Invalid count.\n";
308             }
309             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
310         }
311     }
312
313     void displaydetails() const override
314     {
315         cout << "\n----- Gala Dinner Summary ----- \n";
316         cout << "\nDate: " << date << "\nOrganizer: " << organizer->getName() << endl;
317         for (const auto &va : venuesWithAttendees)
318             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
319     }
320 };
321

```

- The *GalaDinner* class publicly inherits from *Event*, making it another specific type of event.
- It overrides the `inputdetails()` method to gather details for a gala dinner.
- Within `inputdetails()`, it prompts the user to enter the event date in "DD-MM-YYYY" format, validating it using `isValidDate()` and `isAtLeastOneWeekAhead()` from the *Event* class. This loop continues until a valid date is provided.
- It then creates an *Organizer* object using `Organizer::create_organizer()` and assigns it to the inherited `organizer` member.
- The `venuesWithAttendees` vector (inherited from *Event*) is cleared.
- It iterates through the `selectedVenues` (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. `cin.ignore()` is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a *VenueAttendees* object, moving the `unique_ptr` to the `venueCopy` and the attendees count into it, and then adds this object to the `venuesWithAttendees` vector.
- It also overrides the `displaydetails()` method to present a summary of the gala dinner.
- Inside `displaydetails()`, it prints a "Gala Dinner Summary", including the `date`, the `organizer's` name, and then iterates through `venuesWithAttendees` to display each `venue's` name along with its corresponding attendee count.

```

322 // Drive class Distinguished Lecture
323 class DistinguishedLecture : public Event
324 {
325 public:
326     void inputdetails() override
327     {
328         cout << "\n----- Distinguished Lecture Details ----- \n";
329         while (true)
330         {
331             cout << "Enter Lecture Date (DD-MM-YYYY): ";
332             getline(cin, date);
333             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
334                 break;
335             cout << "Invalid format or date must be at least one week ahead.\n";
336         }
337         organizer = Organizer::create_organizer();
338         venuesWithAttendees.clear();
339         for (const auto &venue : selectedVenues)
340         {
341             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
342             int attendees;
343             while (true)
344             {
345                 cout << "Attendees at " << venueCopy->getName() << ": ";
346                 cin >> attendees;
347                 cin.ignore();
348                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
349                     break;
350                 cout << "Invalid count.\n";
351             }
352             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
353         }
354     }
355
356     void displaydetails() const override
357     {
358         cout << "\n----- Distinguished Lecture Summary ----- \n";
359         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
360         for (const auto &va : venuesWithAttendees)
361             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
362     }
363 };
364

```

- The *DistinguishedLecture* class publicly inherits from Event, making it another specific type of event.
- It overrides the `inputdetails()` method to gather details for a distinguished lecture.
- Within `inputdetails()`, it prompts the user to enter the lecture date in "DD-MM-YYYY" format, validating it using `isValidDate()` and `isAtLeastOneWeekAhead()` from the Event class. This loop continues until a valid date is provided.
- It then creates an *Organizer* object using `Organizer::create_organizer()` and assigns it to the inherited organizer member.
- The `venuesWithAttendees` vector (inherited from Event) is cleared.
- It iterates through the `selectedVenues` (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. `cin.ignore()` is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a *VenueAttendees* object, moving the `unique_ptr` to the `venueCopy` and the attendees count into it, and then adds this object to the `venuesWithAttendees` vector.
- It also overrides the `displaydetails()` method to present a summary of the distinguished lecture.
- Inside `displaydetails()`, it prints a "Distinguished Lecture Summary", including the date, the organizer's name, and then iterates through `venuesWithAttendees` to display each venue's name along with its corresponding attendee count.

```
365 // Drive class Research Colloquium
366 class ResearchColloquium : public Event
367 {
368 public:
369     void inputdetails() override
370     {
371         cout << "\n----- Research Colloquium Details ----- \n";
372         while (true)
373         {
374             cout << "Enter Date (DD-MM-YYYY): ";
375             getline(cin, date);
376             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
377                 break;
378             cout << "Invalid format or date must be at least one week ahead.\n";
379         }
380         organizer = Organizer::create_organizer();
381         venuesWithAttendees.clear();
382         for (const auto &venue : selectedVenues)
383         {
384             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
385             int attendees;
386             while (true)
387             {
388                 cout << "Attendees at " << venueCopy->getName() << ": ";
389                 cin >> attendees;
390                 cin.ignore();
391                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
392                     break;
393                 cout << "Invalid count.\n";
394             }
395             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
396         }
397     }
398
399     void displaydetails() const override
400     {
401         cout << "\n----- Research Colloquium Summary ----- \n";
402         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
403         for (const auto &va : venuesWithAttendees)
404             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
405     }
406 };
407
```

- The **ResearchColloquium** class publicly inherits from Event, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for a research colloquium.
- Within **inputdetails()**, it prompts the user to enter the event date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the Event class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited organizer member.
- The **venuesWithAttendees** vector (inherited from Event) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the research colloquium.
- Inside **displaydetails()**, it prints a "Research Colloquium Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.

```

408 // Drive class Academic Workshop
409 class AcademicWorkshop : public Event
410 {
411 public:
412     void inputdetails() override
413     {
414         cout << "\n----- Academic Workshop Details ----- \n";
415         while (true)
416         {
417             cout << "Enter Date (DD-MM-YYYY): ";
418             getline(cin, date);
419             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
420                 break;
421             cout << "Invalid format or date must be at least one week ahead.\n";
422         }
423         organizer = Organizer::create_organizer();
424         venuesWithAttendees.clear();
425         for (const auto &venue : selectedVenues)
426         {
427             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
428             int attendees;
429             while (true)
430             {
431                 cout << "Attendees at " << venueCopy->getName() << ": ";
432                 cin >> attendees;
433                 cin.ignore();
434                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
435                     break;
436                 cout << "Invalid count.\n";
437             }
438             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
439         }
440     }
441
442     void displaydetails() const override
443     {
444         cout << "\n----- Academic Workshop Summary ----- \n";
445         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
446         for (const auto &va : venuesWithAttendees)
447             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
448     }
449 };
450

```

- The **AcademicWorkshop** class publicly inherits from Event, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for an academic workshop.
- Within **inputdetails()**, it prompts the user to enter the event date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the Event class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited organizer member.
- The **venuesWithAttendees** vector (inherited from Event) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the academic workshop.
- Inside **displaydetails()**, it prints an "Academic Workshop Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.

```

451 // Drive class Guest Speaker Session
452 class GuestSpeakerSession : public Event
453 {
454 public:
455     void inputdetails() override
456     {
457         cout << "\n----- Guest Speaker Session Details ----- \n";
458         while (true)
459         {
460             cout << "Enter Date (DD-MM-YYYY): ";
461             getline(cin, date);
462             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
463                 break;
464             cout << "Invalid format or date must be at least one week ahead.\n";
465         }
466         organizer = Organizer::create_organizer();
467         venuesWithAttendees.clear();
468         for (const auto &venue : selectedVenues)
469         {
470             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
471             int attendees;
472             while (true)
473             {
474                 cout << "Attendees at " << venueCopy->getName() << ": ";
475                 cin >> attendees;
476                 cin.ignore();
477                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
478                     break;
479                 cout << "Invalid count.\n";
480             }
481             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
482         }
483     }
484
485     void displaydetails() const override
486     {
487         cout << "\n----- Guest Speaker Session Summary ----- \n";
488         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
489         for (const auto &va : venuesWithAttendees)
490             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
491     }
492 };
493

```

- The **GuestSpeakerSession** class publicly inherits from **Event**, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for a guest speaker session.
- Within **inputdetails()**, it prompts the user to enter the session date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the **Event** class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited **organizer** member.
- The **venuesWithAttendees** vector (inherited from **Event**) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the guest speaker session.
- Inside **displaydetails()**, it prints a "Guest Speaker Session Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.

```

494 // Drive class Career Fair
495 class CareerFair : public Event
496 {
497 public:
498     void inputdetails() override
499     {
500         cout << "\n----- Career Fair Details ----- \n";
501         while (true)
502         {
503             cout << "Enter Date (DD-MM-YYYY): ";
504             getline(cin, date);
505             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
506                 break;
507             cout << "Invalid format or date must be at least one week ahead.\n";
508         }
509         organizer = Organizer::create_organizer();
510         venuesWithAttendees.clear();
511         for (const auto &venue : selectedVenues)
512         {
513             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
514             int attendees;
515             while (true)
516             {
517                 cout << "Attendees at " << venueCopy->getName() << ": ";
518                 cin >> attendees;
519                 cin.ignore();
520                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
521                     break;
522                 cout << "Invalid count.\n";
523             }
524             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
525         }
526     }
527
528     void displaydetails() const override
529     {
530         cout << "\n----- Career Fair Summary ----- \n";
531         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
532         for (const auto &va : venuesWithAttendees)
533             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
534     }
535 };
536

```

- The **CareerFair** class publicly inherits from Event, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for a career fair.
- Within **inputdetails()**, it prompts the user to enter the career fair date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the Event class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited organizer member.
- The **venuesWithAttendees** vector (inherited from Event) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the career fair.
- Inside **displaydetails()**, it prints a "Career Fair Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.


```
537 // Drive class Debate Competition
538 class DebateCompetition : public Event
539 {
540 public:
541     void inputdetails() override
542     {
543         cout << "\n----- Debate Competition Details ----- \n";
544         while (true)
545         {
546             cout << "Enter Date (DD-MM-YYYY): ";
547             getline(cin, date);
548             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
549                 break;
550             cout << "Invalid format or date must be at least one week ahead.\n";
551         }
552         organizer = Organizer::create_organizer();
553         venuesWithAttendees.clear();
554         for (const auto &venue : selectedVenues)
555         {
556             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
557             int attendees;
558             while (true)
559             {
560                 cout << "Attendees at " << venueCopy->getName() << ": ";
561                 cin >> attendees;
562                 cin.ignore();
563                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
564                     break;
565                 cout << "Invalid count.\n";
566             }
567             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
568         }
569     }
570
571     void displaydetails() const override
572     {
573         cout << "\n----- Debate Competition Summary ----- \n";
574         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
575         for (const auto &va : venuesWithAttendees)
576             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
577     }
578 };
579
```

- The *DebateCompetition* class publicly inherits from Event, making it another specific type of event.
- It overrides the `inputdetails()` method to gather details for a debate competition.
- Within `inputdetails()`, it prompts the user to enter the competition date in "DD-MM-YYYY" format, validating it using `isValidDate()` and `isAtLeastOneWeekAhead()` from the Event class. This loop continues until a valid date is provided.
- It then creates an *Organizer* object using `Organizer::create_organizer()` and assigns it to the inherited organizer member.
- The `venuesWithAttendees` vector (inherited from Event) is cleared.
- It iterates through the `selectedVenues` (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. `cin.ignore()` is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a *VenueAttendees* object, moving the `unique_ptr` to the `venueCopy` and the attendees count into it, and then adds this object to the `venuesWithAttendees` vector.
- It also overrides the `displaydetails()` method to present a summary of the debate competition.
- Inside `displaydetails()`, it prints a "Debate Competition Summary", including the date, the organizer's name, and then iterates through `venuesWithAttendees` to display each venue's name along with its corresponding attendee count.


```

580 // Drive class Graduation Ball
581 class GraduationBall : public Event
582 {
583 public:
584     void inputdetails() override
585     {
586         cout << "\n----- Graduation Ball Details ----- \n";
587         while (true)
588         {
589             cout << "Enter Date (DD-MM-YYYY): ";
590             getline(cin, date);
591             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
592                 break;
593             cout << "Invalid format or date must be at least one week ahead.\n";
594         }
595         organizer = Organizer::create_organizer();
596         venuesWithAttendees.clear();
597         for (const auto &venue : selectedVenues)
598         {
599             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
600             int attendees;
601             while (true)
602             {
603                 cout << "Attendees at " << venueCopy->getName() << ": ";
604                 cin >> attendees;
605                 cin.ignore();
606                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
607                     break;
608                 cout << "Invalid count.\n";
609             }
610             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
611         }
612     }
613
614     void displaydetails() const override
615     {
616         cout << "\n----- Graduation Ball Summary ----- \n";
617         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
618         for (const auto &va : venuesWithAttendees)
619             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
620     }
621 };
622

```

- The **GraduationBall** class publicly inherits from Event, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for a graduation ball.
- Within **inputdetails()**, it prompts the user to enter the ball date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the Event class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited organizer member.
- The **venuesWithAttendees** vector (inherited from Event) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the graduation ball.
- Inside **displaydetails()**, it prints a "Graduation Ball Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.

```
623 // Drive class Poster Presentation Session
624 class PosterPresentationSession : public Event
625 {
626 public:
627     void inputdetails() override
628     {
629         cout << "\n----- Poster Presentation Session Details ----- \n";
630         while (true)
631         {
632             cout << "Enter Date (DD-MM-YYYY): ";
633             getline(cin, date);
634             if (isValidDate(date) && isAtLeastOneWeekAhead(date))
635                 break;
636             cout << "Invalid format or date must be at least one week ahead.\n";
637         }
638         organizer = Organizer::create_organizer();
639         venuesWithAttendees.clear();
640         for (const auto &venue : selectedVenues)
641         {
642             auto venueCopy = make_unique<Venue>(venue->getName(), venue->getCapacity());
643             int attendees;
644             while (true)
645             {
646                 cout << "Attendees at " << venueCopy->getName() << ": ";
647                 cin >> attendees;
648                 cin.ignore();
649                 if (attendees >= 0 && attendees <= venueCopy->getCapacity())
650                     break;
651                 cout << "Invalid count.\n";
652             }
653             venuesWithAttendees.push_back({std::move(venueCopy), attendees});
654         }
655     }
656
657     void displaydetails() const override
658     {
659         cout << "\n----- Poster Presentation Session Summary ----- \n";
660         cout << "Date: " << date << "\nOrganizer: " << organizer->getName() << endl;
661         for (const auto &va : venuesWithAttendees)
662             cout << "Venue: " << va.venue->getName() << ", Attendees: " << va.attendeeCount << endl;
663     }
664 };
665
```

- The **PosterPresentationSession** class publicly inherits from **Event**, making it another specific type of event.
- It overrides the **inputdetails()** method to gather details for a poster presentation session.
- Within **inputdetails()**, it prompts the user to enter the session date in "DD-MM-YYYY" format, validating it using **isValidDate()** and **isAtLeastOneWeekAhead()** from the **Event** class. This loop continues until a valid date is provided.
- It then creates an **Organizer** object using **Organizer::create_organizer()** and assigns it to the inherited **organizer** member.
- The **venuesWithAttendees** vector (inherited from **Event**) is cleared.
- It iterates through the **selectedVenues** (a collection of pre-selected venues) to obtain attendee counts for each. For each venue, it prompts for the number of attendees, validates that the count is non-negative and does not exceed the venue's capacity. **cin.ignore()** is used to clear the input buffer after reading the integer attendees.
- After valid attendee input, it creates a **VenueAttendees** object, moving the **unique_ptr** to the **venueCopy** and the attendees count into it, and then adds this object to the **venuesWithAttendees** vector.
- It also overrides the **displaydetails()** method to present a summary of the poster presentation session.
- Inside **displaydetails()**, it prints a "Poster Presentation Session Summary", including the date, the organizer's name, and then iterates through **venuesWithAttendees** to display each venue's name along with its corresponding attendee count.

```
666 // ----- Event::create_event Implementation -----
667 void Event::create_event(int event_number)
668 {
669     int choice;
670     cout << "\n===== Select Event Type For Event #" << event_number << " =====\n";
671     cout << "1. Convocation Ceremony\n"
672           << "2. Gala Dinner\n"
673           << "3. Distinguished Lecture\n"
674           << "4. Research Colloquium\n"
675           << "5. Academic Workshop\n"
676           << "6. Guest Speaker Session\n"
677           << "7. Career Fair\n"
678           << "8. Debate Competition\n"
679           << "9. Graduation Ball\n"
680           << "10. Poster Presentation Session\n";
681     cout << "Enter The Event Type Number That You Want To Create: ";
682     cin >> choice;
683     cin.ignore();
684
685     while (choice < 1 || choice > 10)
686     {
687         cout << "Invalid choice. Please select a number between 1 and 10: ";
688         cin >> choice;
689         cin.ignore();
690     }
691
692     // Let user select venues first
693     select_venues();
694
695     unique_ptr<Event> event;
696
697     switch (choice)
698     {
699     case 1:
700     {
701         event = make_unique<ConvocationCeremony>();
702         break;
703     }
704     case 2:
705     {
706         event = make_unique<GalaDinner>();
707         break;
708     }
709     }
```

- The **create_event** static method takes an integer **event_number** as input, which is used for display purposes to indicate which event is being created.
- It presents the user with a menu of ten different event types, ranging from **"Convocation Ceremony"** to **"Poster Presentation Session."**
- The user is prompted to enter a number corresponding to the desired event type, and input is validated to ensure it's between 1 and 10. **cin.ignore()** is used to clear the input buffer.
- Before creating the event, it calls **select_venues()**, which is a utility function (presumably a member of the class containing **create_event**) to allow the user to select venues for the event.
- A **unique_ptr<Event>** event is declared to hold the newly created event object, leveraging polymorphism.
- A switch statement then uses the user's choice to dynamically create an object of the corresponding event type (e.g., ConvocationCeremony, GalaDinner, etc.) using **make_unique** and assigns it to the event **unique_ptr**. This structure is likely continued for all 10 event types.
- The method then returns this **unique_ptr<Event>**, providing a generic pointer to the specific event type created, facilitating unified handling of different event types through their common Event base class interface.

- This segment of code continues the switch statement from the **create_event** method, handling choices 3 through 10.
- For each case, it dynamically allocates a **unique_ptr** to a specific derived Event type (e.g., `DistinguishedLecture`, `ResearchColloquium`, `AcademicWorkshop`, `GuestSpeakerSession`, `CareerFair`, `DebateCompetition`, `GraduationBall`, `PosterPresentationSession`).
- Each **make_unique** call creates an object of the chosen event type, and the **unique_ptr** is assigned to the event variable declared earlier in the **create_event** method.
- The **break** statement after each case ensures that only the code block corresponding to the user's choice is executed.
- This entire switch structure embodies a factory pattern, allowing the **create_event** method to produce different concrete Event objects based on user input, while returning them as a generic **unique_ptr<Event>**.

```

709     case 3:
710     {
711         event = make_unique<DistinguishedLecture>();
712         break;
713     }
714     case 4:
715     {
716         event = make_unique<ResearchColloquium>();
717         break;
718     }
719     case 5:
720     {
721         event = make_unique<AcademicWorkshop>();
722         break;
723     }
724     case 6:
725     {
726         event = make_unique<GuestSpeakerSession>();
727         break;
728     }
729     case 7:
730     {
731         event = make_unique<CareerFair>();
732         break;
733     }
734     case 8:
735     {
736         event = make_unique<DebateCompetition>();
737         break;
738     }
739     case 9:
740     {
741         event = make_unique<GraduationBall>();
742         break;
743     }
744     case 10:
745     {
746         event = make_unique<PosterPresentationSession>();
747         break;
748     }

```

- The **default** case in the switch statement handles any choice that is not between 1 and 10. If an invalid choice is made, it prints **"Invalid choice."** and immediately returns, preventing further execution within the method.
- After the switch statement, if a valid event event (as a **unique_ptr<Event>**) has been created, the **event->inputdetails()** method is called **polymorphically**. This means the specific **inputdetails()** implementation for the actual derived event type (e.g., **ConvocationCeremony::inputdetails()**, **GalaDinner::inputdetails()**) will be invoked.

```

749     default:
750     {
751         cout << "Invalid choice.\n";
752         return;
753     }
754
755     event->inputdetails();
756     events.push_back(move(event));
757 }
758
759

```

- Finally, **events.push_back(move(event))** adds the newly created event to a collection of events (presumably `vector<unique_ptr<Event>> events`) using **std::move**. This transfers ownership of the **unique_ptr** from the local event variable to the events vector, ensuring proper memory management and preventing the event object from being prematurely destroyed when **create_event** exits.

```
760 // ----- Main -----
761 int main()
762 {
763     cout << "=====Event Management System=====\\n";
764
765     cout << "How many events do you want to create? ";
766     int count;
767     cin >> count;
768     cin.ignore();
769
770     for (int i = 0; i < count; i++)
771     {
772         Event::create_event(i + 1);
773     }
774
775     cout << "\\n===== Event Summary =====\\n";
776     for (const auto &e : events)
777     {
778         e->displaydetails();
779         cout << "-----\\n";
780     }
781
782     return 0;
783 }
```

- The **main** function is the entry point of the program, initiating an "Event Management System."
- It first prompts the user to enter the number of events they wish to create, reads this count, and then clears the input buffer using **cin.ignore()**.
- A for loop iterates count times, calling **Event::create_event(i + 1)** in each iteration. This means for each event to be created, the **create_event** static method (which handles user input for event type, date, organizer, and attendees) is invoked.
- After all events have been created, the program prints an "Event Summary" heading.
- It then iterates through a collection named events (presumably a **std::vector<std::unique_ptr<Event>>** that stores all the created event objects).
- For each event **e** in the events collection, it **polymorphically** calls **e->displaydetails()**, which in turn invokes the appropriate **displaydetails()** method for the specific type of event (e.g., **ConvocationCeremony::displaydetails()**, **GalaDinner::displaydetails()**).
- Finally, the program returns 0, indicating successful execution.

Sample Output

```
=====Event Management System=====
How many events do you want to create? 1

===== Select Event Type For Event #1 =====
1. Convocation Ceremony
2. Gala Dinner
3. Distinguished Lecture
4. Research Colloquium
5. Academic Workshop
6. Guest Speaker Session
7. Career Fair
8. Debate Competition
9. Graduation Ball
10. Poster Presentation Session
Enter The Event Type Number That You Want To Create: 6

===== Venue Selection =====
Available Venues:
1. Auditorium (Capacity: 200)
2. Sports Complex (Capacity: 1000)
3. Grand Hall (Capacity: 800)
4. Ballroom (Capacity: 500)
5. Hotel Banquet Hall (Capacity: 1000)
6. Grand Dining Hall (Capacity: 400)
7. Lecture Theatre (Capacity: 300)
8. Conference Room (Capacity: 500)
9. Seminar Room (Capacity: 100)
10. Departmental Auditorium (Capacity: 150)
11. Classroom (Capacity: 35)
12. Computer Lab (Capacity: 40)
13. Exhibition Space (Capacity: 300)
14. Marquee/Outdoor Event Space (Capacity: 500)
How many venues do you want to select (1-14)? 1
Select Venue #1: 3
```

```
----- Guest Speaker Session Details -----
Enter Date (DD-MM-YYYY): 20-06-2025
Enter Organizer Name: Islamic Ideology Council
Attendees at Grand Hall: 700

===== Event Summary =====

----- Guest Speaker Session Summary -----
Date: 20-06-2025
Organizer: Islamic Ideology Council
Venue: Grand Hall, Attendees: 700
-----
```

THE END
